



KERNEL SYSTEMS SECURITY · TERM PROJECT

MAKIM

Multi-Agent Kernel Integrity Monitor

Linux Rootkit Detection via AI agents & eBPF

Taha Yusuf Gandhi · 20266094

Agenda

01 Motivation & Problem Statement

02 MAKIM Overview

03 The Snapshot Analysis Pipeline

04 The Runtime Monitoring Pipeline

05 Implementation Environment

06 Validation Results

07 Performance & Comparison

08 Limitations & Conclusion

Why Kernel Security Matters?



Physical AI Systems

Robots & autonomous vehicles run on Linux kernels. A compromised kernel can manipulate sensor data or motor controls.



AI Inference Servers

Model weights live in RAM. Ring-0 rootkit can silently exfiltrate a sensitive model or manipulate inference results.



Multi-Agent Communications

Agent-to-agent messages pass through the kernel's network stack. A rootkit can intercept and alter them invisibly.



Cloud Infrastructure

Every VM, container, and serverless function depends on the host kernel. One rootkit = entire tenant isolation gone.



A **Ring-0 rootkit** can do all of this while remaining completely invisible to every user-space tool. Existing tools (rkhunter, chkrootkit) rely on static signatures and fail to reason across combinations of “weak signals” or unknown threats.

The Problem — Existing Tools Fall Short

Section 1

Tool	Detection Method	Zero-Day?	AI Reasoning?	Remediation?	Main Gap
rkhunter	MD5 hash + signature DB	✗ No	✗ No	✗ No	Fails on any unknown rootkit
chkrootkit	String pattern matching	✗ No	✗ No	✗ No	Very high false-positive rate
Tripwire	File integrity (inotify)	Partial	✗ No	✗ No	No kernel memory visibility
OSSEC	Log correlation rules	Partial	✗ No	✗ No	Rules need manual authoring
MAKIM ✓	Multi-agent + LLM	✓ Heuristic	✓ OpenRouter	✓ Yes	User-space ceiling only

The core problem: all existing tools detect what they were programmed to detect. None can reason about combinations of weak signals, correlate events over time, or explain what to do next.

MAKIM — Design Philosophy

MAKIM has 4 operational modes

01

Flag : --baseline

Captures clean system state and saves it to JSON baseline. Runs this first.
Uses Agent 1 (scanner_agent)

03

Flag : --live

Real-time kernel event stream monitoring
Uses Agent 6 (live_sentinel_agent)

02

(Default) 5-Agent model

Compare current state vs baseline + LLM Analysis.
Uses Agent 1-5

04

Flag : -live-demo

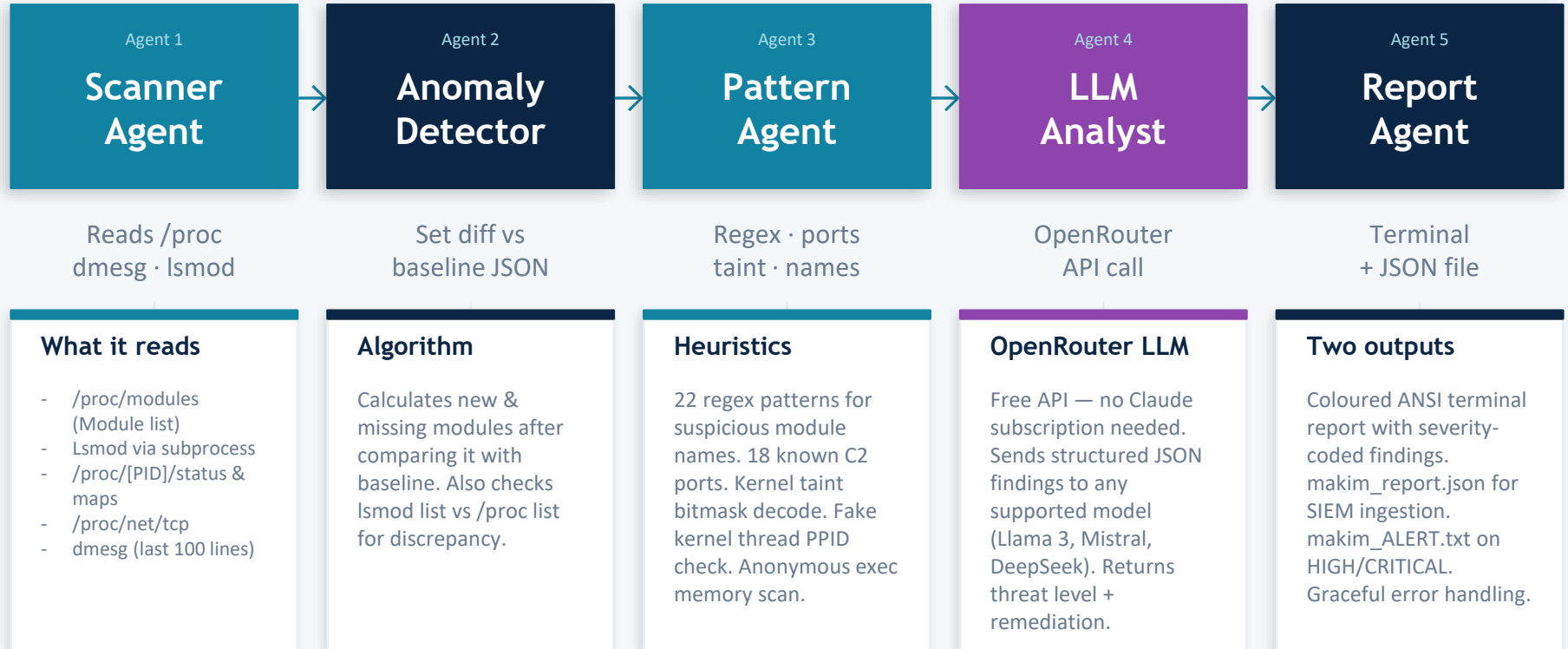
Live monitor + automated safe demo activity
Uses Agent 6 (live_sentinel_agent)

Result: a 15ms point-in-time scanner that detects DKOM, syscall hooking, C2 connections, and shellcode — with AI-generated remediation steps.

The 5-Agent Pipeline

Architecture Overview

 Linux Kernel Interface — /proc/modules · /proc/[PID]/* · dmesg · lsmod · /proc/net/tcp · /proc/kallsyms



Agent 1 — Scanner Agent

Collecting Raw Kernel State



`/proc/modules`



`lsmod command`



`/proc/[PID]/status`



`/proc/[PID]/maps`



`/proc/net/tcp`



`dmesg`

⚡ All reads are READ-ONLY. No kernel modules needed. Runs entirely in user-space. Completes in ~12ms on a 57-process system.

The Baseline Diffing Algorithm

- 1 Load baseline.json
- 2 Build Python sets
- 3 Set subtraction: NEW
- 4 Set subtraction: MISSING
- 5 Ismod discrepancy check

Anomaly Types Detected

NEW_MODULE

Module in current /proc/modules not in baseline

HIGH

MISSING_MODULE

Baseline module no longer loaded

MEDIUM

MODULE_HIDDEN_FROM_LSMOD

/proc/modules has it but lsmod doesn't → syscall hook

CRITICAL

NEW_PROCESS

Process name not present at baseline time

MEDIUM

POTENTIAL_DKOM

>5 PIDs with /proc folder but unreadable status

HIGH

NEW_TCP_CONNECTION

Outbound ESTABLISHED connection not in baseline

MEDIUM

Suspicious Module Names

- 22 compiled regex patterns applied to every loaded module name
- Patterns include: ^hide, ^rootkit, ^rk_, keylog, backdoor, stealth, syscall_hook
- Known-legit prefix whitelist (ext4, nvidia, usb...) reduces false positives

Known C2 Port Detection

- 18 hardcoded malware ports: 4444 (Metasploit), 31337 (elite), 1337, 6666, 6667
- Checks BOTH local and remote sides of every ESTABLISHED/LISTEN connection
- Ports 22, 80, 443 are whitelisted via the allowlist so no false alarms.

Anonymous Executable Mappings

- Reads /proc/[PID]/maps for ALL running processes
- Flags regions where: permissions contain 'x' AND filename is empty/[anon]
- Normal code loads from files (/usr/lib/libc.so.6). Shellcode has no file.
- Excludes [stack], [vdso], [vsyscall] — these are legitimately anonymous+exec

Kernel Taint + Process Anomalies

- Reads /proc/sys/kernel/tainted as a bitmask (each bit = a different taint type)
- Bit 2 = module force-loaded, Bit 11 = unsigned module → both flagged as HIGH
- Fake kernel thread: process with [brackets] name but PPID ≠ 2 (kthreadd)
- Real kernel threads ALWAYS have PPID=2. Any deviation = impersonation attempt

AGENT 4 — LLM ANALYST

LLM agent is integrated with **OpenRouter**'s free model API that aggregates findings from Agents 1-3.

Response Schema Requirements :



Threat Level : CLEAN to CRITICAL



Confidence : LOW/MED/HIGH



Remediation : Sequential steps



IoCs : Network/Process indicators

If OFFLINE :

- **Fallback** mechanism designed
- Set rules given to score the trust level based on types of anomalies found.
- Sets level as CRITICAL if score > 80 & HIGH > 60
- AI analysis part and Remediation part is predefined.



Prompt Structure Sent to OpenRouter

```
=== MAKIM Security Scan Report ===
```

```
Baseline available: True/False
```

```
Kernel modules loaded: 57
```

```
Processes running: 57
```

```
TCP connections: 8
```

```
--- ANOMALY DETECTOR FINDINGS ---
```

```
Overall severity: CRITICAL
```

```
[1] [CRITICAL] MODULE_HIDDEN_FROM_LSMOD:
```

```
    Module 'rkmodule' in /proc but not lsmod
```

```
[2] [HIGH] NEW_MODULE: 'rkmodule'
```

```
[3] [HIGH] POTENTIAL_DKOM: 20 PIDs
```

```
--- PATTERN FINDINGS ---
```

```
[1] [HIGH] SUSPICIOUS_PORT: port 4444
```

```
[2] [HIGH] FAKE_KERNEL_THREAD: [kworker/evil]
```

```
Please analyze and return JSON assessment.
```

1



Terminal Report

Colour-coded using 5 severity levels: CRITICAL=bright red, HIGH=red, MEDIUM=yellow, LOW=cyan, CLEAN=green. Includes emoji indicators, wrapped text for long descriptions, and section dividers to make it user-readable. Consists of threat level, AI analysis & Remediation steps as well.

```
===== MAKIM Report =====  
THREAT LEVEL: 🚨 HIGH  
[CRITICAL] MODULE_HIDDEN_FROM_LSMOD  
[HIGH] POTENTIAL_DKOM (20 PIDs)
```

2



makim_report.json

Complete machine-readable report with fixed schema. Includes: threat level, AI analysis, all anomalies with severity, pattern findings, raw module list, TCP connections, baseline timestamp, etc.

```
"threat_level": "HIGH",  
"ai_summary": "...",  
"anomaly_detection": {...},  
"pattern_detection": {...}
```

3



makim_ALERT.txt

Written ONLY when threat_level ≥ HIGH. Contains: timestamp, threat level, summary, and ordered remediation steps from the LLM. Can be monitored by any file-watching tool to trigger immediate notifications.

```
MAKIM SECURITY ALERT  
Threat: HIGH  
Timestamp: 2026-05-22  
Summary: ...  
1. Isolate system  
2. Memory forensics
```

NEW — AGENT 6

Why Point-in-Time Scanning Is Not Enough?

Agents 1–5 see what HAS changed since baseline. Agent 6 sees what IS changing — right now, at the syscall layer.

What is eBPF?

Extended Berkeley Packet Filter — runs sandboxed programs inside the Linux kernel at syscall tracepoints. Safer than a kernel module: the kernel verifies the program before running it. No crash risk, no kernel patch needed.

What is bpftrace?

A high-level eBPF tracing language. MAKIM writes the bpftrace script dynamically inside `_build_bpftrace_script()` and runs it as a subprocess. Install with: `sudo apt install bpftrace`.

Why Agent 6 is essential

A rootkit can load, act, and unload between two 60-second scans with zero detection. Agent 6 fires an alert the INSTANT a module load syscall (`sys_enter_finit_module`) is invoked — no scan interval gap.

Readiness & Graceful Fallback

Checks: Linux platform, root (`os.geteuid()==0`), bpftrace in PATH (`shutil.which`). If any check fails, prints install guide and saves a readiness report — the snapshot pipeline is never interrupted.

Agent 6 — bpftrace Tracepoints & Trust Scoring Algorithm

How Events Are Captured & Scored

6 Syscall Tracepoints Watched in Real Time

`sys_enter_finit_module`
`sys_enter_init_module`

→ **MODULE_LOAD_ATTEMPT**

-40 pts

Both syscalls load kernel modules. Any rootkit entering the kernel triggers one of these immediately.

`sys_enter_delete_module`

→ **MODULE_UNLOAD_ATTEMPT**

-30 pts

Attackers unload security modules to weaken defences or erase their own traces.

`sys_enter_openat`
`sys_enter_openat2`

→ **SENSITIVE_FILE_OPEN**

-15 pts

Filtered to 3 paths only:
`/proc/kallsyms`,
`/proc/modules`,
`/proc/sys/kernel/tainted`
— rootkit recon targets.

`sys_enter_connect`

→ **NETWORK_CONNECT**

-2 pts

Low penalty — high-volume normal traffic. Trusted processes (NetworkManager, snapd) get zero penalty.

Per-Process Trust Scoring

Start: 100 — Every process begins TRUSTED

TRUSTED

Score 80-100

Normal or low-risk activity

WATCH

Score 50-79

Some suspicious indicators; monitor

SUSPICIOUS

Score 25-49

Multiple events; investigate promptly

HIGH_RISK

Score 0-24

Module-load detected; act immediately

Demo Result (--live-demo):

```
python3 opens /proc/kallsyms, /proc/modules,  
/proc/sys/kernel/tainted + TCP connect  
→ Score = 53 | Label: WATCH
```

Implementation Environment

Operating System



Ubuntu 24.04.4 LTS (Noble Numbat)

Kernel



Linux 6.18.5 #2 SMP PREEMPT_DYNAMIC x86_64

Python



Python 3.12.3 — standard library only

External Packages



NONE — zero pip installs for core functionality

Privilege



User-space daemon — root recommended for full /proc access

Execution Context



Linux container (virtualised kernel — /proc/modules unavailable)

Container Constraint:

The validation environment is a Linux container — /proc/modules and lsmod are not exposed. So kernel module scanning returned 0 modules. All other agents (process scanning, TCP monitoring, dmesg, pattern matching, baseline diffing) worked fully. On a bare-metal Ubuntu 24.04 host, full module scanning including the lsmod cross-check is available.

Validation — Test 1 & 2

Real Outputs from Running MAKIM on Live System

✅ Test 1 — Clean System Baseline Scan

Procedure: Capture baseline, then immediately re-scan on the same unmodified system.

Expected: 0 anomalies, CLEAN threat level.

```
$ python3 main.py --baseline
[Agent 1] Found 0 modules, 56 processes
[Agent 2] Baseline saved: makim_baseline.json
Captured: 2026-05-22T10:28:21

$ python3 main.py
[Agent 1] Found 0 modules, 57 processes
[Agent 2] No anomalies detected
[Agent 3] No known rootkit patterns detected
[Agent 4] No API key — skipping
THREAT LEVEL: ✅ CLEAN
```

Result: PASS — clean scan returns CLEAN with zero false positives

🟡 Test 2 — Live New Process Detection

Procedure: After baseline, start a new background process. Run scan.

Expected: NEW_PROCESS MEDIUM anomaly detected.

```
$ python3 main.py
[Agent 1] Found 0 modules, 59 processes
[Agent 2] 1 anomaly: 0 CRITICAL 0 HIGH 1 MEDIUM
[MEDIUM] NEW_PROCESS
Process 'grep' was NOT in baseline.
Investigate if expected.
[Agent 3] No patterns detected
THREAT LEVEL: 🟢 LOW
(MEDIUM anomaly but no HIGH/CRITICAL)
```

Result: PASS — new process detected. Severity correctly MEDIUM not HIGH.

💡 Design note: Not every new process is malicious. MEDIUM = investigate; HIGH/CRITICAL = act immediately. This tiered severity is intentional.

Validation — Test 3: Simulated Compromise

5 Rootkit Indicators Injected → All 5 Detected

Procedure: Inject a synthetic 'compromised' snapshot directly into the Anomaly Detector. Contains 5 planted rootkit artifacts.

HIGH

NEW_MODULE

Module 'rkmodule' not in baseline

→ Flag for investigation

CRITICAL

MODULE_HIDDEN_FROM_LSMOD

rkmodule in /proc but ABSENT from lsmod — syscall table hook confirmed

→ Isolate system immediately

MEDIUM

NEW_PROCESS

'c2_agent' process not present at baseline

→ Kill and investigate binary

HIGH

POTENTIAL_DKOM

20 PIDs with /proc folder but unreadable status — process hiding

→ Memory forensics required

MEDIUM

NEW_TCP_CONNECTION

192.168.1.5:12345 → 1.2.3.4:6666 — C2 channel not in baseline

→ Block outbound & trace

Result: 5/5 planted anomalies detected · Overall severity = CRITICAL · Test: PASS

The MODULE_HIDDEN_FROM_LSMOD finding correctly escalates to CRITICAL — the highest level — because a discrepancy between /proc/modules and lsmod is definitive proof of syscall table hooking.

Validation — Test 4: Signature Pattern Detection

5 Known Rootkit Patterns → All
5 Detected as HIGH

Procedure: Inject synthetic snapshot with 5 classic rootkit indicators into the Pattern Agent only. No baseline needed.

HIGH

SUSPICIOUS_MODULE_NAME

Input: Module name: 'rootkit_hook'

→ Regex '^rootkit' matches. Module whitelisted? No — not in KNOWN_LEGIT_PREFIXES.

Why: Rootkits disguise themselves as kernel modules. Naming patterns like 'rootkit_', 'hide_', 'syscall_hook' are red flags.

HIGH

SUSPICIOUS_DMESG

Input: dmesg line: 'WARNING: rootkit module detected in memory'

→ Keyword 'rootkit' in SUSPICIOUS_DMESG_KEYWORDS list. Match found.

Why: Rootkits during testing or development sometimes log their own activity. Kernel-level debug messages persist in dmesg ring buffer.

HIGH

SUSPICIOUS_PORT (×2)

Input: ESTABLISHED connection on ports 4444 and 54321

→ 4444 in SUSPICIOUS_PORTS set (Metasploit default). 54321 in set (generic backdoor).

Why: These ports have essentially no legitimate use. Metasploit's default listener is 4444. Seeing it ESTABLISHED = active C2 connection.

HIGH

FAKE_KERNEL_THREAD

Input: Process '[kworker/evil]' with PPID=1234

→ Name has brackets → kernel-thread-style. PPID=1234 ≠ 2 (kthreadd). Contradiction.

Why: All real Linux kernel threads have PPID=2. A process with brackets but wrong parent is impersonating a kernel thread to avoid suspicion.

Result: 5/5 pattern indicators detected as HIGH · Test: PASS · Zero false positives on this synthetic dataset

Validation Summary & Performance Measurements

All 4 Tests Passed · 15ms Scan Time

Test	Scenario	Expected Result	Actual Result	Status
T1	Clean system scan	CLEAN, 0 findings	CLEAN, 0 findings	PASS
T2	New process (grep)	MEDIUM anomaly	MEDIUM anomaly	PASS
T3	5 simulated rootkit artifacts	CRITICAL + 4 more	CRITICAL + 4 more	PASS
T4	5 signature patterns	5× HIGH	5× HIGH	PASS

Performance — Measured on Ubuntu 24.04.4 / Kernel 6.18.5

Agent 1 — Scanner	0.012s	57 processes × /proc/[PID]/status + /proc/net/tcp + dmesg 100 lines
Agent 2 — Anomaly Detector	0.001s	Python set arithmetic — O(1) lookup regardless of list size
Agent 3 — Pattern Agent	0.002s	22 regex patterns × module count + 18 port checks + taint read
Agents 1–3 Total	0.015s	Complete detection pipeline without LLM — runs in 15 milliseconds
Agent 4 — OpenRouter LLM	3–8s*	HTTP POST to openrouter.ai + response parse (est. — no API key in test)

* Agent 4 is optional. Without OpenRouter API key, total pipeline runs in 15ms. LLM adds 3–8s for AI reasoning and remediation advice.

MAKIM — Multi-Agent Kernel Integrity Monitor | 20266094 Taha Yusuf Gandhi | 20 / 25

Detection Coverage — What MAKIM Catches

10 Rootkit Techniques × 3 Detection Layers

Detection Technique	Agent 2 Anomaly	Agent 3 Pattern	Agent 4 LLM Corr.
Kernel module injection	✓	✓	✓
Syscall table hook (via lsm diff)	✓	—	✓
DKOM process hiding	✓	—	✓
C2 outbound connection	✓	✓	✓
Shellcode (anon exec map)	—	✓	✓
Fake kernel thread name	—	✓	✓
Kernel taint (unsigned module)	—	✓	✓
dmesg rootkit keyword	—	✓	✓
New user-space process	✓	—	✓
New listening port (backdoor)	✓	✓	✓

Fundamental

L1 — User-Space Ceiling

All data comes through the kernel's /proc interface. A rootkit that hooks the VFS read path can return falsified data to MAKIM. This is an architectural constraint, not a bug — solving it requires eBPF or a kernel module.

Fix: Complement with eBPF-based monitors (bcc-python, Falco) that operate inside the kernel at kprobe hookpoints below the VFS hook level.

Security

L3 — Unsigned Baseline

The baseline JSON is a plain file. A rootkit with write access can modify it to add its own module/process names, permanently hiding itself from Anomaly Detector findings.

Fix: Sign with HMAC-SHA256 using a hardware-derived key. Verify signature on every load. The cryptography library is already installed.

Architectural

L2 — Point-in-Time Only

MAKIM scans on demand. A rootkit that loads, exfiltrates, and unloads between two 60-second scans leaves no trace. There is currently no continuous monitoring or event-driven detection.

Fix: Add --daemon mode with asyncio scan loop + inotify on /lib/modules/ for real-time module load detection.

Deployment

L4 — DKOM Heuristics

The PID-folder vs status-readability DKOM heuristic produces false positives from race conditions (process exits between listing and reading) and false negatives against rootkits that also hide /proc folder entries

Fix: True DKOM detection requires reading kernel memory directly or using eBPF to hook the task_struct unlink operation in real time.

MAKIM...

- ❖ MAKIM proves that multi-agent AI architectures are viable for Linux kernel integrity monitoring within user-space constraints.
- ❖ Cross-references independent kernel data paths (e.g. lsmod vs /proc/modules) to expose hooks.
- ❖ Scans for anonymous executable memory, fake kernel threads, and malicious C2 connections.
- ❖ The dual pipeline architecture allows snapshot analysis in just 15ms and utilizes sandboxed eBPF to continuously monitor kernel syscalls in real-time.
- ❖ It combines 6 AI agents and eBPF monitoring to detect stealthy Ring 0 rootkits and provides LLM-guided analysis and remediation.



THANK YOU